Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 6

Implementation of Error Detection using Checksum and CRC

Name:	Vijayaraaghavan K S
Register Number:	3122247001066
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	19 August 2026

1. Objective

To develop a socket programming application in C that simulates reliable data transmission using two error-detection techniques:

- **Checksum:** data is divided into 8-bit words, summed using 1's complement addition with end-around carry, and the resulting checksum is verified at the receiver.
- **Cyclic Redundancy Check (CRC):** data is treated as a binary polynomial, divided by a generator polynomial $G(x) = x^4 + x + 1$ (i.e. `10011`), and the remainder (CRC) is appended to the frame before transmission over TCP sockets. Bit errors are then deliberately introduced to demonstrate that CRC verification correctly detects them.

2. Server Algorithm

1. Create a TCP socket, bind to port 8080, and `listen()` for connections.
2. `accept()` each client connection and spawn a detached thread to service it concurrently.
3. Read one byte identifying the mode: `'C'` for Checksum or `'R'` for CRC.
4. `recv()` the payload, where the final byte is the checksum/CRC appended by the sender and the preceding bytes are the data.
5. **Checksum mode:** recompute the 1's-complement checksum of the received data (`checksum_verify()`) and compare it against the received checksum byte.
6. **CRC mode:** recompute the CRC of the received data using the same generator polynomial `10011` (`crc_calc()`) and compare it against the received CRC byte.
7. Print the received vs. calculated value and declare the frame `ERROR-FREE` if they match, or `ERROR DETECTED` otherwise.

3. Client Algorithm

1. Ask the user to choose Checksum or CRC mode, then read the data string to transmit.
2. **Checksum mode:** sum each byte into a running 16-bit accumulator, printing the intermediate sum after every byte; fold the carry out of the top byte back into the sum (end-around carry); take the 1's complement of the result as the final checksum.
3. **CRC mode:** process the data one byte at a time with an 8-bit polynomial-division register: XOR each byte into the register, then shift left 8 times, XOR-ing in the generator polynomial (`0x13` = `10011`) whenever the top bit is 1; the register's final value is the CRC remainder.
4. Connect to the server over TCP, send the mode byte, then send the data with the checksum/CRC appended as the last byte.
5. **CRC mode only:** optionally flip one bit of the already-sent frame at a user-chosen byte position and resend it to the server unmodified except for that corruption, to demonstrate that the receiver's recomputed CRC no longer matches.

4. Server Code

4.1 server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>

#define PORT 8080
#define GENERATOR 0x13

typedef struct { int sock; char type; } client_t;

unsigned char checksum_verify(char *data, int len) {
    unsigned int sum = 0;
    for (int i = 0; i < len; i++) sum += (unsigned char)data[i];
    while (sum >> 8) sum = (sum & 0xFF) + (sum >> 8);
    return (unsigned char)(~sum & 0xFF);
}

unsigned char crc_calc(char *data, int len) {
    unsigned char crc = 0;
    for (int i = 0; i < len; i++) {
        crc ^= data[i];
        for (int j = 0; j < 8; j++) {
            if (crc & 0x80) crc = (crc << 1) ^ GENERATOR;
            else crc <<= 1;
            crc &= 0xFF;
        }
    }
    return crc;
}

void *client_handler(void *arg) {
    client_t *client = (client_t *)arg;
    int sock = client->sock;
    char type = client->type;
    free(client);

    char buffer[1024];
    int bytes_read = recv(sock, buffer, sizeof(buffer) - 1, 0);

    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';
        printf("\n[Server] Received: %s (%d bytes)\n", buffer, bytes_read);

        if (type == 'C') {
            unsigned char recv_checksum = buffer[bytes_read - 1];
            unsigned char calc_checksum = checksum_verify(buffer, bytes_read - 1);
            printf("[CHECKSUM] Received: 0x%02X, Calculated: 0x%02X\n",
                recv_checksum, calc_checksum);
            printf("[Result] %s\n", recv_checksum == calc_checksum ? "ERROR-FREE" :
                 "ERROR DETECTED");
        } else {
            unsigned char recv_crc = buffer[bytes_read - 1];
            unsigned char calc_crc = crc_calc(buffer, bytes_read - 1);
            printf("[CRC] Received: 0x%02X, Calculated: 0x%02X\n", recv_crc,
                calc_crc);
            printf("[Result] %s\n", recv_crc == calc_crc ? "ERROR-FREE" : "ERROR
                DETECTED");
        }
    }
```

```
    close(sock);
    return NULL;
}

int main() {
    int server_fd, client_sock;
    struct sockaddr_in addr;
    int addrlen = sizeof(addr);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("[Server] Listening on port %d...\n", PORT);

    while (1) {
        client_sock = accept(server_fd, (struct sockaddr *)&addr, (socklen_t*)&
            addrlen);
        if (client_sock >= 0) {
            char choice;
            recv(client_sock, &choice, 1, 0);

            pthread_t tid;
            client_t *c = malloc(sizeof(client_t));
            c->sock = client_sock;
            c->type = choice;

            pthread_create(&tid, NULL, client_handler, c);
            pthread_detach(tid);
        }
    }
    close(server_fd);
    return 0;
}
```

5. Client Code

5.1 client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define SERVER_IP "127.0.0.1"
#define GENERATOR 0x13

unsigned char checksum_calc(char *data, int len) {
    unsigned int sum = 0;
    printf("[Intermediate Sum Calculation]\n");
    for (int i = 0; i < len; i++) {
        sum += (unsigned char)data[i];
```

```
        printf("  Step %d: 0x%02X + Sum = 0x%04X\n", i + 1, (unsigned char)data[i],
            sum);
    }
    printf("  Total Sum: 0x%04X\n", sum);

    unsigned int carry = sum >> 8;
    sum = sum & 0xFF;
    printf("  Carry (End-around): 0x%02X\n", carry);

    sum += carry;
    printf("  After adding carry: 0x%04X\n", sum);

    unsigned char checksum = (unsigned char)(~sum & 0xFF);
    printf("  1's Complement (Checksum): 0x%02X\n", checksum);
    return checksum;
}

unsigned char crc_calc(char *data, int len) {
    unsigned char crc = 0;
    printf("[CRC Calculation with G(x)=10011]\n");
    for (int i = 0; i < len; i++) {
        crc ^= data[i];
        for (int j = 0; j < 8; j++) {
            if (crc & 0x80) crc = (crc << 1) ^ GENERATOR;
            else crc <<= 1;
            crc &= 0xFF;
        }
    }
    printf("  Final CRC: 0x%02X\n", crc);
    return crc;
}

void demo_checksum() {
    char data[1024];
    printf("\n=== CHECKSUM MODE ===\n");
    printf("Enter data: ");
    fgets(data, sizeof(data), stdin);
    data[strcspn(data, "\n")] = 0;

    int len = strlen(data);
    unsigned char checksum = checksum_calc(data, len);

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &addr.sin_addr);

    if (connect(sock, (struct sockaddr *)&addr, sizeof(addr)) >= 0) {
        send(sock, "C", 1, 0);
        char packet[1025];
        strcpy(packet, data);
        packet[len] = checksum;
        send(sock, packet, len + 1, 0);
        printf("[Sent] Data with Checksum (0x%02X)\n", checksum);
        sleep(1);
    }
    close(sock);
}

void demo_crc() {
    char data[1024];
    printf("\n=== CRC MODE ===\n");
```

```
    printf("Enter data: ");
    fgets(data, sizeof(data), stdin);
    data[strcspn(data, "\n")] = 0;

    int len = strlen(data);
    unsigned char crc = crc_calc(data, len);

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &addr.sin_addr);

    if (connect(sock, (struct sockaddr *)&addr, sizeof(addr)) >= 0) {
        send(sock, "R", 1, 0);
        char packet[1025];
        strcpy(packet, data);
        packet[len] = crc;
        send(sock, packet, len + 1, 0);
        printf("[Sent] Data with CRC (0x%02X)\n", crc);

        printf("\nDemonstrate error detection? (y/n): ");
        char choice;
        scanf("%c", &choice);
        if (choice == 'y' || choice == 'Y') {
            printf("Introduce bit error (flip bit at position 0-7): ");
            int pos;
            scanf("%d", &pos);
            if (pos >= 0 && pos < len) {
                packet[pos] ^= 0x01;
                printf("[Modified] Bit flipped at position %d\n", pos);

                int sock2 = socket(AF_INET, SOCK_STREAM, 0);
                connect(sock2, (struct sockaddr *)&addr, sizeof(addr));
                send(sock2, "R", 1, 0);
                send(sock2, packet, len + 1, 0);
                printf("[Sent] Modified Data (Error Injected)\n");
                sleep(1);
                close(sock2);
            }
        }
        sleep(1);
    }
    close(sock);
}

int main() {
    printf("Error Detection Methods:\n1. Checksum\n2. CRC\nChoose (1/2): ");
    int choice;
    scanf("%d\n", &choice);

    if (choice == 1) demo_checksum();
    else if (choice == 2) demo_crc();
    else printf("Invalid choice\n");

    return 0;
}
```

6. Output / Test Cases

Three cases were exercised: (1) Checksum on the string `HELLO`, verified error-free; (2) CRC on the string `TEST`, verified error-free; (3) the same CRC frame re-sent with one bit of the first byte flipped, which the server correctly flags as `ERROR DETECTED`.

6.1 Client Output – Checksum Mode (HELLO)

```
Error Detection Methods:
1. Checksum
2. CRC
Choose (1/2):
=== CHECKSUM MODE ===
Enter data: [Intermediate Sum Calculation]
  Step 1: 0x48 + Sum = 0x0048
  Step 2: 0x45 + Sum = 0x008D
  Step 3: 0x4C + Sum = 0x00D9
  Step 4: 0x4C + Sum = 0x0125
  Step 5: 0x4F + Sum = 0x0174
  Total Sum: 0x0174
  Carry (End-around): 0x01
  After adding carry: 0x0075
  1's Complement (Checksum): 0x8A
[Sent] Data with Checksum (0x8A)
```

6.2 Client Output – CRC Mode with Injected Bit Error (TEST)

```
Error Detection Methods:
1. Checksum
2. CRC
Choose (1/2):
=== CRC MODE ===
Enter data: [CRC Calculation with G(x)=10011]
  Final CRC: 0xF1
[Sent] Data with CRC (0xF1)

Demonstrate error detection? (y/n): Introduce bit error (flip bit at position 0-7):
     [Modified] Bit flipped at position 0
[Sent] Modified Data (Error Injected)
```

6.3 Server-side Verification Log

```
[Server] Listening on port 8080...

[Server] Received: HELLO (6 bytes)
[CHECKSUM] Received: 0x8A, Calculated: 0x8A
[Result] ERROR-FREE

[Server] Received: TEST (5 bytes)
[CRC] Received: 0xF1, Calculated: 0xF1
[Result] ERROR-FREE

[Server] Received: UEST (5 bytes)
[CRC] Received: 0xF1, Calculated: 0xF6
[Result] ERROR DETECTED
```

Flipping bit 0 of the character `'T'` (`0x54`) changed it to `'U'` (`0x55`), which the receiver's Wireshark-independent, application-level CRC check correctly identified: the received CRC (`0xF1`, computed by the sender over the original data) no longer matches the CRC the receiver recalculates over the corrupted data (`0xF6`).

7. Learning Outcomes

- Implemented 1's-complement checksum arithmetic with end-around carry from first principles, and observed step-by-step how the carry-out of the high byte must be folded back into the sum before complementing.
- Implemented binary polynomial division (CRC) using an 8-bit shift-and-XOR register driven by the generator polynomial $G(x) = x^4 + x + 1$, and understood why the choice of generator determines which burst-error patterns are detectable.
- Verified experimentally that CRC reliably detects single-bit corruption, whereas a naive recomputation and comparison is only meaningful if both sender and receiver use the exact same generator polynomial and byte framing.
- Practised structuring an error-detection scheme as an application-layer protocol over TCP, with the detection code appended to the payload as a trailer, mirroring how data-link layer frames carry an FCS/CRC field.
- Gained experience building a multi-mode (checksum/CRC) client-server system that shares a single TCP port and thread-per-client server design.